第 12 章　现代的通用计算机

在本书的开头，我们的目标是制造一台计算机。但是，到目前为止，我们只发明了一台全自动运行的加法机。

自动加法机也是计算机，不用怀疑，我这么说一点都没错。尽管它不能用来听音乐、看电影、玩游戏和上网种菜，但是你看，它有存储器，有运算器（不好意思，只能做加法），还有一个奇妙的控制器，甚至包括一个用来驱动控制器的心脏，那就是振荡器——所有这一切，正如你已经听别人说过的那样，都是组成一台现代的计算机所必不可少的。

把指令放到存储器里，然后加以执行，这是冯·诺依曼先生的主意，为此很多人称他为计算机之父。但是，诺依曼先生很谦虚，他告诉大家说，计算机的基本概念属于图灵。

阿兰·图灵（1912—1954）是英国的数学家和逻辑学家，他是天才，也是怪物，从生活中的某些方面来看，他常常显得一无是处。1936 年，他在 24 岁的时候，在他一篇论文中提出了“图灵机”的理论。

千万不要误会，我的意思并不是说图灵先生写了一篇论文，告诉世人应当如何来造一台叫做“图灵机”的计算机。不是这样的。尽管图灵和诺依曼生活在同一个时代，但前者的精力更多的是花在数学和逻辑上。事实上，那篇论文说的是另一件事情，讨论的主题是逻辑的完备性，即所有数学问题是否在原则上都是可解的。作为论证过程的一部分，他提出了图灵机的概念。谁也没有想到，这个小小的构想会引起世人的关注，闪耀出那么夺目的光芒。

要想把图灵机是怎么回事说清楚不是件容易的事儿，得先从 9 世纪说起，讲一讲波斯数学家阿勒·霍瓦里松和他的《代数对话录》，告诉大家什么是算法；然后，作为一个实例，我们再回到古希腊，看看如何用欧几里得算法，通过固定的步骤来得到两个数的最大公约数；最后，我们回到近代的 1900 年。在那一年举行的世界数学家大会上，德国数学家戴维·希尔伯特提出了一个有关逻辑完备性的问题，即是否所有的数学问题在原则上都是可解的。于是图灵先生写了上面那篇论文，并用一个图灵机的模型作为注解，回答了那个问题，即有些数学问题是不可解的。

说实在的，我很想把图灵机讲清楚，可这肯定又是长篇大论。尤其是考虑到我这个人比较愚钝，数学成绩不好，从小只知道上树掏鸟、下河摸鱼，八岁了还从一数不到一百，所以只好作罢。

图灵毫无疑问是一位杰出的理论家，但是在生活中，在面临实际问题的时候，却显得非常不切实际、愚钝甚至可笑，这在他小时候就已经表露无遗。他在办公室里把啤酒罐拴在散热器上防止同事们饮用，而他们很自然地把这当成一种挑战，撬开锁，然后恣意饮用。有时候为了赶赴一个约会，他常常步行好几十公里而不用任何交通工具，尽管每次都累得不行，但却从不迟到。1939年，当第二次世界大战爆发的时候，他将自己的全部积蓄换成两个大银块埋在乡下以保证安全，但战争结束之后却忘了埋在哪里。

图灵机从来没有成为一台真正的机器，它是想象出来的，但却给同时代的人以启发。在我们生活的这个星球上，每天都有无数的计算机在亮着，每天都有无数的计算机被生产出来，这一台台的机器，大部分都采用了冯·诺依曼的设计，所以称为冯·诺依曼体系结构。不可否认的是，冯·诺依曼的设计实际上可以看成是图灵机的一个简单实现。

图灵的思想为现代计算机的设计指明了方向。现在，我们唯一要做的就是按照这种思路来继续完善我们的自动加法机，来看看现代计算机大体上是怎么工作的，以及它如此有用的原因。

12.1 更多的计算机指令

在第11章里，我们已经发明了一台能够自动运行的加法机。最重要的是，它是依靠执行指令来工作的，这就使它成了一台真正的计算机，因为现代的计算机就是按照这种方式工作的。当然，就像一个只有五六岁的孩子，它现在还很简单。

指令的执行是一个有趣、巧妙的过程。正如我们已经看到的，在计算机的内部有着各种各样的小东西——计数器、译码器、加法器、传输门、寄存器，等等，要使计算机能够按这些器件发出的指令工作，必须巧妙地安排连线，并预先设计好各种指令所需要的操作序列。然后，在振荡器那有节律的跳动下，所有相关器件都应该在恰当的时机“动”一下。就这样，数据从一个地方出来，经过不同的器件，最后变成另一种形式，回到另一个目的地。

从另一个角度来看，让计算机执行指令就像在法国用法语点菜、在英国用英语点菜。所以，把指令看做是计算机的语言或者母语，是很贴切的。

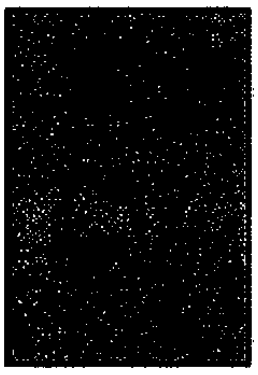
从先贤们发明了计算机开始，围绕它工作着两类人，一类人关心的是如何制造计算机的实体，也就是实实在在能看得见的东西，这称为硬件；另一类人不太在意计算机内部是怎么工作的，他们只想知道硬件能执行哪些指令，并把这些指令编排到一起来做某件事，这个过程称为编程，编排好的指令称为程序。这是很贴切的，在现代汉语里，“程序”的意思是事情进行的先后次序，编排指令也是一

样，先用哪条指令，先计算什么，是分步骤按顺序来的。相应的，负责编排程序的家伙们称为“程序员”。

在第 11 章里，我们讲了全自动加法计算机。从程序员的角度来看，这台机器的内部构造是次要的。在他们眼里，这台机器只有三样东西有价值。第一样，是存储器，因为他们需要在这里编程；第二样，是加法器，因为它是数学计算实际进行的地方，是扑克牌里的皇后。

最后一样对程序员来说尤其重要，它就是寄存器 RA。原因很简单，他们可以不知道指令是怎么执行的，但绝对要知道机器把执行的结果放到哪里了。

如图 12.1 所示，这是程序员眼中的计算机。很显然，尽管在计算机的内部还有很多寄存器，但它们都是隐姓埋名的临时寄存器，只有 RA 是头面人物。造成这种差别的，是身份问题：谁有资格出现在计算机指令中。



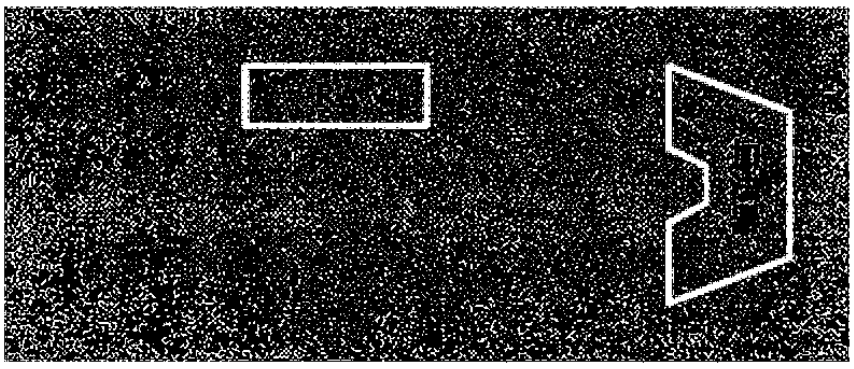


图 12.1　我们已知的两条指令只和寄存器 RA 打交道。
对程序员来说，计算机的其他内部细节并不重要

比如，我们已经拥有了两条和数学计算有关的指令，即装载和相加，它们都需要寄存器 RA。对于装载指令来说，它只是需要 RA 中有一个我们想要的数字；而对于相加指令来说，RA 既是两个相加的数字之一，又是加法结果的归宿，否则，真是不敢想象，因为你不知道计算机把结果放在什么地方，所以也没办法找到它，这不是计算机的义务。

作为一个例子，我们把装载指令的操作码定为 10001，它指示了三层意思。第一，这是个装载数字的指令；第二，目的地是寄存器 RA；第三，该指令还包括一个操作数，它位于操作码后面的下一个存储单元里。

所以，你不要小看了这个“二进制数”，它不但让你知道要做什么，结果到哪里找，还包含了足够的信息让计算机开始干活儿。如图 12.2 所示，第一条指令的操作码为 10001，操作数是 00110，表示要把数字 00110 装载到寄存器 RA 中。

一旦执行这条指令，寄存器 RA 里的数字就是 00110，也就是十进制的 6。换句话说，当该指令执行的时候，装载到寄存器 RA 中的数字直接来自于指令本身，是该指令的组成部分，可以从指令中立即得到，所以称为立即数。

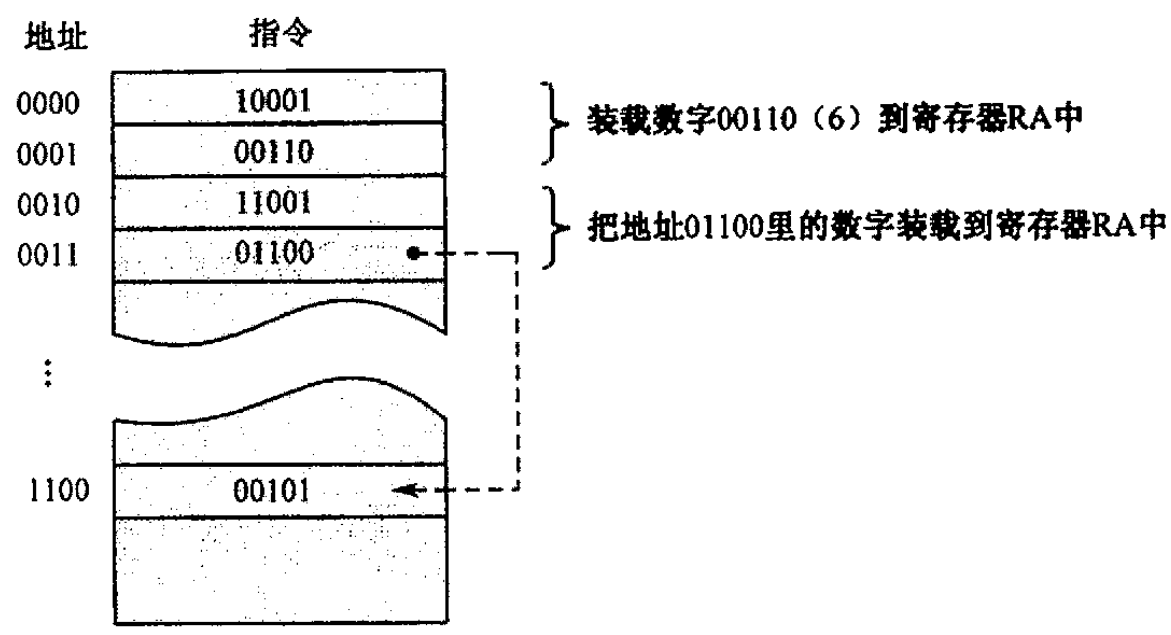


图 12.2　两种不同的装载指令，它们的工作方式大相径庭

在本书之外的所有其他书里，装载指令被称为传送指令，实际上它们是一回事。但是，不管是装载，还是传送，都未能充分表达指令本身的功能，因为东西一旦传送出去，就少了一件，而指令所做的却是在目的地复制一份。我知道没有人喜欢和二进制打交道，为了方便地书写指令，装载——不，传送指令——需要一个简单易行的表示方法，我觉得这种方法不错：

MOV RA, 16

“MOV”是英语单词“MOVE”的缩写，本身就有“传送”的意思。这条指令表示把立即数 16 传送到寄存器 RA 中。既然这种写法是给我们人类看的，就没必要采用二进制数，十进制更直观。

人类的思维真是奇怪，一旦说到“立即数”，我们就会想到“非立即数”、“间接数”或者“拐弯抹角数”。没错，确实有这样的情况，指令要操作的数字并非来自于指令本身，但是指令却告诉我们到哪里能找到这个数字。

如图 12.2 所示，同样是装载数字的指令，但它却具有另一个不同的操作码 11001，这是因为它需要向控制器表达另外三层意思。第一，这是个装载指令；第二，目的地是寄存器 RA；第三，跟在操作码后面的不是一个立即数，而是另一个存储单元的地址！

因为这条指令的操作数是 01100，所以它指向存储器的另一个地址 1100（十进制的 12），而这个地址里存放的是数字 00101，也就是十进制的 5，所以当这条指令执行完毕的时候，寄存器 RA 中的数字是 5。与第 11 章里那台简单的自动加法机不同，我们需要添加额外的硬件来把该指令的操作数作为地址赋给地址寄存器 AR，并再次访问存储器以取得实际的数字。

尽管同样是装载指令，为了表明它的操作数是一个地址，需要另外一种标志，比如一对括号：

MOV RA, (12)

通常情况下，计算机存储器的容量都足够大，不但可以存放程序指令，还可以留出一部分来存放数据。因此，如果我们在编写程序的时候，不在指令中使用立即数，而代之以间接的地址，就不用老是把程序改来改去，或者重新编排，只需要每次把那个地址里换成不同的数字就可以了。

存储器有足够的空间，一旦我们决定要开垦它，就会发现这里需要更多的指令（以及更复杂的硬件设计）。比如，可以将寄存器 RA 中的数字传送到某个存储单元里。当然，你需要在指令中指定一个地址：

MOV (25), RA

执行这条指令，将把 RA 中的数字传送到地址为 25 的存储单元里。甚至可以直接把一个立即数传送到指定的存储单元里：

MOV (30), 20

尽管表面上看不出来，但这条指令是我们所见过的最复杂的，因为在操作码后面跟了两个操作数，一个是存储单元地址，另一个是立即数。

最后，关于传送指令我们要说的是，有些计算机不允许在两个存储单元之间传送数据。所以，在那种计算机上，像这样的指令是不存在的：

MOV (23), (18)

理由很简单，在两个存储单元之间传送数据可以用寄存器中转（即使你不这样做，计算机在内部也会用临时寄存器来中转），这样可以节省硬件成本并减小设计难度。

说完了传送指令，再来说说相加指令。我们实际上已经拥有一条这样的指令，在那里，寄存器 RA 中的数字和指令中的立即数（比如 25）相加，结果返回 RA 中。这条指令可以表示成：

ADD RA, 25

在英语里，“ADD”的意思是把一个数和另一个数相加。在指令中直接指定立即数不是个好主意，因为一旦数字变了，就要重写指令。为了获得一些灵活性，有必要将相加操作和存储单元建立关联。比如，可以用某个存储单元里的数字同寄存器 RA 中的数字相加：

ADD RA, (12)

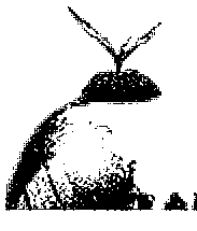
指令执行完毕，寄存器 RA 中将获得相加的结果。不过，要是你想把结果放到存储单元里，则可以换一条指令：

ADD (12), RA

像寄存器一样，存储单元也可以直接和指令中的立即数相加。当然，这又需要一条指令：

ADD (22), 9

在某些类型的计算机上，同样不允许两个存储单元之间直接做加法。



和存储器有关的指令使我们得以充分利用它的空间，我认为对它最好的感谢就是写更多更好的程序。比如，为了将任意两个数相加而不用重写程序，我们会编写如图 12.3 那样的指令。

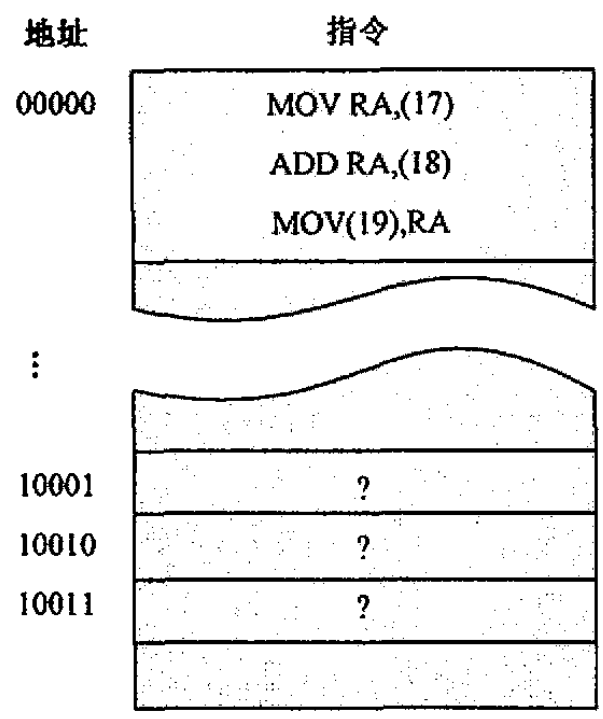


图 12.3　通过在指令中使用地址，即使参与计算的数字变了，也不用重新编写程序

稍微费点心思，这几条指令的意思是不难理解的，尤其是当你知道二进制数 10001、10010、10011 分别等于十进制数 17、18、19 的时候。很明显，程序写好之后就不用再动了，每当你有两个数字相加时，就把它们分别放在地址为 17 和 18 的存储单元里，然后让机器从头开始执行，完毕后就能在地址为 19 的存储单元里得到结果。

12.2　当计算机面临选择时

不知道你发现没有，我们在使用加法指令时，有一个潜藏的问题，或者说隐患。怎么回事呢？你看这条指令：

ADD (22), RA

一般的，存储器的每个存储单元和寄存器具有相同的比特数——换句话说，数据宽度。考虑到我们一直把存储器和寄存器设计成能读/写 5 个比特的，所以现在仍延续这种做法。

5 个比特意味着，每个存储单元，以及寄存器 RA，所能表示的二进制数最大为 11111，也就是十进制的 31。

看上面那条指令，假如地址为 22 的存储单元里保存的是 11001（十进制的 25），寄存器 RA 中保存的是 10110（十进制的 22），那么执行这条指令后，结果就应该是 6 个比特的 101111（十进制的 47）。遗憾的是，为了同存储器和寄存器取得一致，加法器的输出线也是 5 根。在这种情况下，加法器输出的是 01111，并在内部

产生一个进位。所以，当那条指令执行完毕后，地址为22的存储单元里实际上得到的是01111。

这当然是不正确的结果，但眼下我们无能为力，唯一的办法就是重新设计这台计算机，并添加一些新的指令，这些指令可以判断是否产生了进位，然后分别做不同的处理。

比如，我们可以用两个存储单元来保存相加的结果，反正存储器大得很。如图12.4所示，存储器地址10001（17号存储单元）和10010（18号存储单元）里保存的分别是两个要相加的数；地址10100（20号存储单元）用于保存相加的结果（右5位，也称为低5位）。如果相加时产生了进位，则将数字“0”传送到地址10011（19号存储单元），否则就传送一个“1”。这样，当需要显示或者打印的时候，将这两个存储单元归拢到一起就是正确的计算结果。

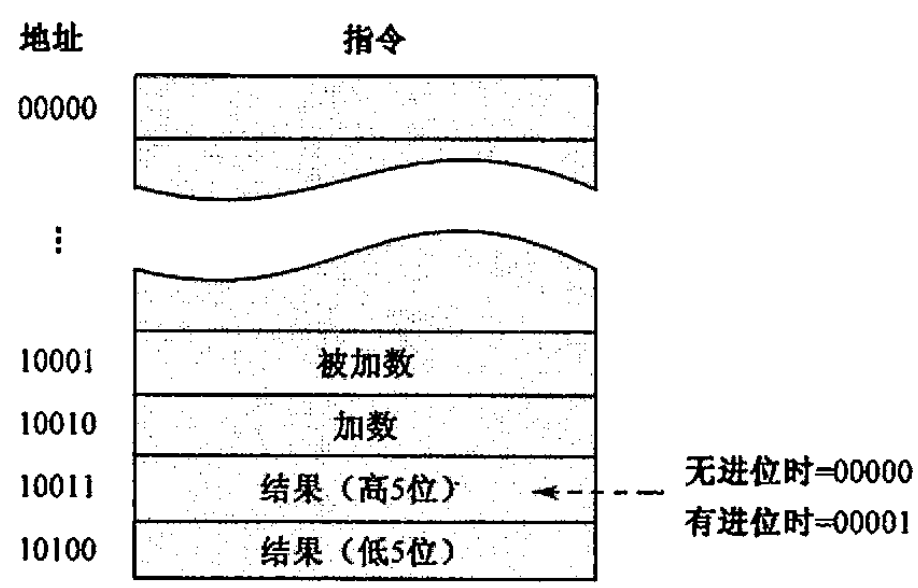


图12.4　两个5位的二进制数相加，结果可能会是6位，所以必须分配2个存储单元

问题是，进位是不可预知的，有些数相加会有，而另一些则没有，你该如何在计算机运行的时候插上一手，或者用放大镜观察到呢？

不需要这样，阿兰·图灵和冯·诺依曼早就已经给出了答案。

在图灵和诺依曼关于运算机器的思想里，有很重要的一点被我们忽略了，那就是每条指令的执行不仅取决于这条指令的目的和功能，还取决于上一条指令执行的结果。比如说，一个聪明的猎人每天可能会“执行”两条“指令”：

听天气预报；
打猎去

取决于听天气预报的结果，如果听来的结果是下雨，那么打猎指令的执行结果就和平日不一样，因为谁也不愿意在雨天出门。

同样的道理，将17、18号存储单元里的数字相加需要以下指令：

MOV RA, (17)
MOV (20), RA
MOV RA, (18)
ADD (20), RA

很好，计算结果的低 5 位已经位于第 20 号存储单元。现在所要做的，就是根据上面最后一条指令：

ADD (20), RA

判断是否产生了进位来进行不同的处理。如果没有产生进位，就把 19 号存储单元写“0”：

MOV (19), 0

否则就写“1”：

MOV (19), 1

我们知道，指令在存储器里的存放是一条接着一条、按顺序来的，而执行的时候也是这样。这就意味着，要解决目前的两难问题，需要一条指令，能根据是否产生了进位来跳转到不同的存储器位置接着执行。

现在麻烦你回到第 6 章，看看图 6.4，两个 3 比特的数相加，结果由 4 根线引出。其中 s_3 是最后一个全加器的进位输出。同样的道理，在我们的加法器里，两个 5 比特的数相加，应该产生 6 比特的输出，但是，第 6 个比特，也就是最后的那个进位，被扔掉了，或者悬空了，没有使用。

这当然是不行的。如图 12.5 所示，我们把加法器的进位线拉出来，接到一个 D 触发器上。这样，每次计算结果出来的时候，如果没有进位，则 Q = 0；否则 Q = 1。

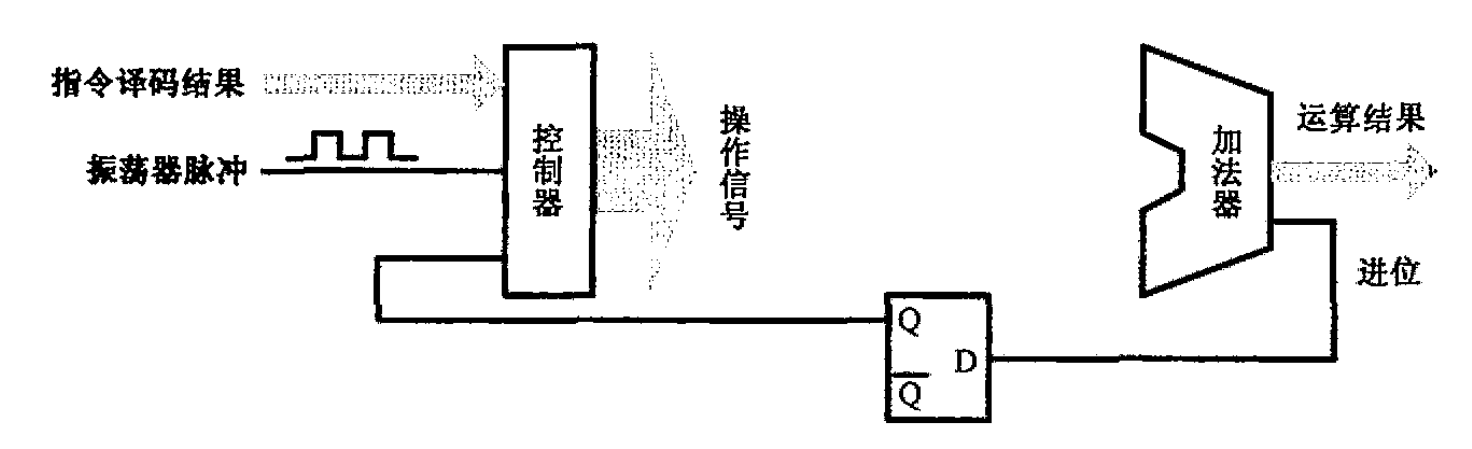


图 12.5　上一条指令的执行状态被送到控制器，将改变后续指令的操作信号

加法器的计算结果有可能会很快消失，但触发器上的进位却能一直保持，直到下一条指令的结果把它挤掉。

从图中可以看出，触发器的 Q 通往整台计算机的控制器。对于控制器来说，这个输入称为进位标志，它和指令译码结果一起，可能改变很多指令的执行过程，如果它们“愿意”被它影响的话。

那么，很好，这正是我们所希望的，因为我们需要根据是否产生进位来跳到其他指令那里去执行。比如：

JC 51

“JC”是“Jump if Carry”的缩写，意思是如果进位则跳转。所以，一旦前面的指令产生了进位，这条指令将使计算机跳到存储器地址为 51 的地方接着往下执行。但是，如果进位实际上没有产生，那这条指令执行后什么也不会发生。

如图 12.6 所示，一旦指令

ADD (20), RA

没有产生进位，那么接下来的指令

JC 12

将什么也不做，计算机接着执行

MOV (19), 0

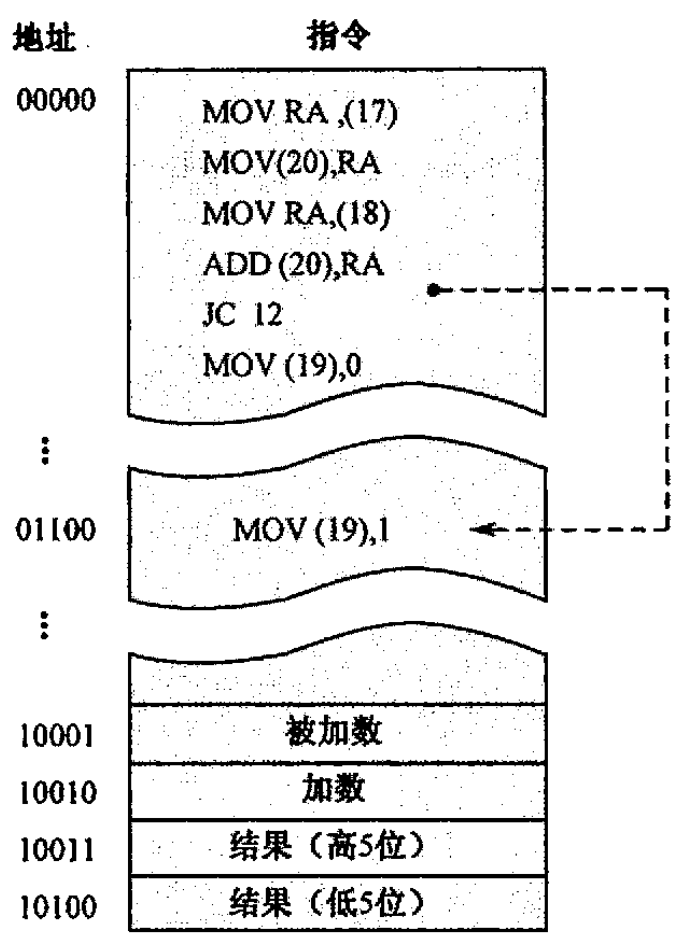


图 12.6 跳转指令示意图

但是，如果有进位产生，那么跳转行为将实际发生，计算机跳转到存储地址 12（即 01100）那里执行，执行的指令是：

MOV (19), 1

看得出，为了解决进位问题，我们兜的圈子可不小。实际上，两个数相加没有这么麻烦，很多计算机都设计了一条带进位加法指令，比如：

ADC RA, 30

这条指令的执行取决于前一条指令。如果前一条指令没有产生进位，那么它

就是单纯的加法，和 ADD 指令没有任何区别；相反的，如果前一条指令产生了进位，那么，它将 RA 的内容和 30 相加之后，还要再加上进位“1”。

所以，前面的进位难题可以很简单地这样来解决：

```
MOV (19), 0
MOV RA, (17)
ADD RA, (18)
ADC (19), 0
MOV (20), RA
```

12.3 现代计算机的大体特征

一般来说，存储器、寄存器和加法器具有相同的数据宽度——用大白话来说，它们的数据引线具有相同的条数。比如说，如果数据线有 8 根，则存储器的每个存储单元包含 8 个比特，寄存器 RA 需要用 8 个上升沿 D 触发器来制造，而加法器呢，则必定是由 8 个全加器组合而成，如图 12.7 所示。换言之，招亲朋，开宴会，有多少个人，就得准备多少双筷子，这个称为计算机的字长。

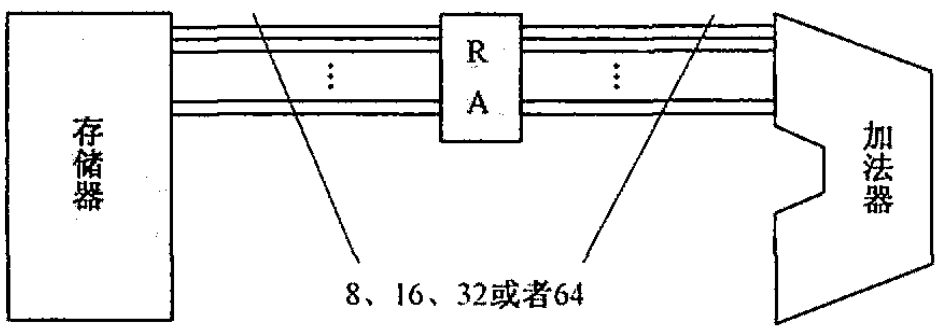


图 12.7 字长

字长表示一台计算机在一次操作中可以处理的二进制比特数。换句话说，就是每个寄存器可以保存的二进制数是几个比特，或者加法器每次计算的二进制数是几个比特。原则上，对计算机的字长没有任何规定和限制，4 比特、6 比特、20 比特，等等，都是可以的。

其实世上本没有路，有一个人走过，后面跟着走的人多了，也就成了路。20 世纪 50 年代的时候，有一个计算机公司决定把它的产品设计成 8 位的字长。这是有原因的，一个二进制数，它可能不单是一个真正的二进制数字，也可能用于代表其他事物，比如 26 个英文字母、阿拉伯数字、大量的标点符号，以及用于向打印机这样的外部设备发送控制命令（如果不是这样的话，你就不能用计算机写报告并把它打印出来，你所写的每一个字，包括标点符号，在计算机内部都和二进制数字一模一样）。经过仔细考虑，最终，他们认为使用 8 位的二进制数就足以包括这一切——不太多，但又刚好够用。

8 位的字长称为字节。由于那种计算机的畅销，这事实上已经成了一个“标准”。最开始的时候，他们把字节拼写成 bite，新鲜了没多久，可能是觉得它容易和已经流行很广的 bit 混淆，于是又改成了 byte，发音听起来像是汉语的“拜特”。

传统上，说到“字节”的时候，大家习惯于用一个大写的字母“B”来表示，以区别于表示“位”的小写的“b”。所以：

1B = 8b

当然，要是心情不错，想换一下形式，下面的换算关系也是正确的：

1byte = 8bit

1 个字节 = 8 位

字长是计算机的一个重要技术指标，也许还是最重要的一个指标。要是它太小，就表明这台计算机每次只能计算很小的数。你想想，对于一个 8 位的计算机来说，它的加数和被加数都不能超过二进制的 11111111，也就是 255，这还没有考虑相加之后的结果可能超过 8 位，这将使寄存器放不下。

不过，这也并不意味着一台 8 位的计算机就不能计算像 1 050 + 7 000 这样的数学题，只是有些麻烦，需要分好几次进行计算（使用 ADD 和 ADC 指令）。

但是，如果我们用一台字长为 16 位的计算机来做这道题，则只需要一次就可以完成。重要的还不是计算过程，因为涉及访问存储器，两次访问存储器比只访问一次要花费成倍的时间。这就好比你每次只能扛 50 斤的东西，150 斤的东西就得分三次扛走；相反，要是你能扛 150 斤的东西，只要扛一次就够了。

江山代有人才出，各领风骚数百年。对于我们平时所用的个人计算机来说，几十年前 8 位是主流（4 位的时候也曾经有过），后来是 16 位。在读这本书的时候，你的计算机可能是当前比较流行的 32 位，也可能是 64 位的，但后者就目前来说还很少。

字长是一个复杂的问题，不是一次可以计算多大的数字那么简单。想想看，在 8 位计算机的时代，一条装载指令将从存储器里取得一个 8 比特的操作数；而在 32 位计算机上，你必须让这条指令仍以 8 位的模式工作[①]，同时，专门为新计算机而设计的指令则需要按 32 位的方式操作，以获得升级换代的好处。在这种情况下，你该怎样设计存储器和寄存器，并如何连线，以使它们井井有条、和谐共存呢？

遗憾的是，要解决这些问题，本书无能为力。不过，这并不是说无法解决，相反，解决得很好，无非是修修补补，你平时在网上种菜所用的计算机就是这么修补来的。就像我们很希望城市里全是高楼，所有的街道都整齐划一，但实际上通常都是房屋参差不齐，街道弯弯曲曲，错综复杂。但是，整个城市还是运转得

① 这称为向后的兼容性。兼容性是必须考虑的问题，为了保证既往的投资，那些老的程序必须继续发挥应有的作用，重新编写这些东西需要继续花钱。

很好，不是吗！

除了字长上的区别，一台计算机只能计算加法也是个问题。所以，我们应当让计算机能够做诸如减法、乘法等这些运算。因此，在造出了加法器之后，再接着制造减法器、乘法器和除法器，等等，这对于已经掌握了数字逻辑学知识的人来说，也并非难事，但已经不是本书所要关心的事情。

你也许期望着我们把平方、开平方根等这些功能都纳入进来，但是要我说，这没必要，因为数学的基本任务是将复杂的、高等的运算转化成基本的加、减、乘、除四则运算。比如，5 的平方实际上可以转化成两个 5 的连乘：

$$5^2 = 5 \times 5$$

所以这台机器只要具备这四种基本运算功能就足够了。通常，这几种运算功能的电路结合在一起，称为运算器。

除了算术计算功能的增强，现代计算机也会增加可以在指令中使用的寄存器数量。比如说，可以在 RA 的基础上，继续增加寄存器 RB、RC 和 RD，如图 12.8 所示。

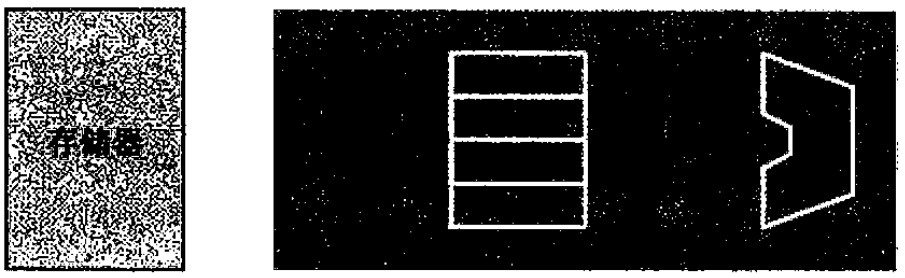


图 12.8　现代计算机的内部会有不止一个寄存器

韩信点兵，多多益善，你可能觉得 4 个还是太少。寄存器多了虽好，但这是以增加计算机内部电路的复杂度为代价的，你需要更多的连线、更多的传输门和更复杂的控制器，制造成本也会迅速增加。所以，现代的计算机都只有少量的寄存器可供使用。

为了用好这些资源，现在唯一要做的就是添加指令。首先，所有的寄存器，以及任何一个存储单元，都可以装载一个包含在指令中的立即数。

其次，任意两个寄存器之间，或者寄存器和存储单元之间，都可以互相装载数据。比如，可以有一条指令将寄存器 RC 中的内容装载（复制）到 RA 中；再比如，可以把寄存器 RB 中的内容装载（复制）到存储器中地址为 225 的存储单元里。

再次，所有的寄存器，以及任何一个存储单元，都可以和包含在指令中的立即数进行加、减、乘、除运算，结果依然返回该寄存器或者存储单元。比如我们前面已经熟悉的相加指令。

最后，寄存器之间，或者寄存器和存储单元之间，都可以互相进行数学运算。

除此之外，现代计算机还会根据一条指令的执行情况产生各种标志，比如我

们已经熟悉的进位标志。其他的标志还包括计算结果为 0、结果中 1 的个数为奇数/偶数，等等。这些标志可以用于跳转指令，或者其他想要参考这些标志的指令。

可以想象，对于现今的任何一台计算机来说，都需要大量的指令支撑它们的运转，用于解决我们所可能碰到的方方面面的问题（但不可能是所有问题，比如吃饭）。对于任何一种类型的计算机来说，它的指令在种类和数量上都是有限的，但是不管有多少，它所能执行的所有指令，称为这种计算机的指令集。

12.4 为什么计算机如此有用

更多的指令和更强有力的控制器能让计算机做越来越多的事情。但是，无论它的功能有多少，我们依然没有摆脱一个尴尬处境，那就是，用机器算数学题更费劲儿，还没有手工做这些事情来得快。想想看，比如下面这道题：

$$(10+3\times6-5)\div5$$

首先，就像我们一直喋喋不休地强调的那样，你不可能指望机器成了精，能够听懂你对它说："题都写在这张纸上了，好好给我算着，我要去钓鱼了。"

在这种情况下，你唯一所能做的，就是坐下来老老实实地编排指令。这个过程完全和你手工做这道题一样，要考虑先做乘除，再做加减，谁和谁相乘，然后再用结果和谁相加，等等。毫不夸张地说，如果一道题连你自己都不会做，那么就不要指望机器会做。毕竟，你必须给出详细的计算步骤，像称职的保姆那样安排好一切，机器所做的，只不过是执行你设计好的步骤。

说到这里，不知道你是否已经明白了我的意思。这是一道非常简单的四则混合运算题，直接用眼睛就能很快得出结果。要是用计算机来算，首先得编排程序指令，然后一个一个地写入存储器，拉下大闸，让机器开始执行，这即使不是一个漫长的过程，也够让人厌烦的。难道这就是我们发明计算机所带来的好处？

遗憾的是，对于这个问题的解答可能不会使你感到高兴。用计算机来算题，你得"侍候"到位，必须用你自己的解题过程来编排程序指令，并写入存储器，这个过程甚至比你自己用笔算题还要麻烦，这是事实，即使是在今天，就是你看这本书的时候也是这样。

不过，看到当今世界范围内的计算机产业如此红火，人们都以会使用计算机而感到兴奋的时候，你也许就不那么沮丧了。但你依然不太明白，为什么会发生这种神奇的事情呢？

首先，我们的一生是发现规律、按规律生活的一生，这就是我们存在的方式。每隔一段时间你就要洗澡，因为你知道不洗澡身上就会痒，这就是规律；科学家之所以能站在领奖台上，是因为他发现了规律。同样，如果有人说你业务很熟，只能说明你也已经发现了规律。最后，计算机之所以有用，仅仅是因为我们只让

它干有规律的事情。这里有一个例子，比如计算两个数和的平方：

$$(5+9)^2$$
$$(3+5)^2$$
$$(7+6)^2$$

如果你像往常一样就事论事，重复地为这三道题各自编排指令，这当然是很麻烦的，更不要说以后还会遇到类似的题目。

事实上，但凡上过初中的人都知道，两个数和的平方对应着一个公式：

$$(a+b)^2=a^2+2ab+b^2$$

即使我们的算术逻辑单元不能计算平方（这并不奇怪，有很多计算机不提供这种运算），也没有关系，因为上面的式子可以继续展开：

$$(a+b)^2=a\times a+2\times a\times b+b\times b$$

为了取得灵活性，我们在存储器里专门开辟出几个存储单元，用于存放这两个数 a、b，以及计算结果。即使你不知道 a 和 b 是多少，也没有关系，因为我们的程序将从这些地址里取得实际的数字，然后进行计算。

一旦这些指令编排完毕，无论你想计算哪两个数和的平方，所要做的仅仅是把那两个数字分别写入存储器的固定位置，然后命令机器开始计算，而不用重复编写程序。一次编写，重复使用，这就是你能获得的好处。当然，对于解释计算机为什么有用，这个例子显得太小，不够分量，不过我们的目的是说明问题即可。

我们都知道圆周率π=3.141 59……这是一个没完没了的数，据说它的小数部分已经借助于计算机算到了万亿位，也有很多人热衷于通过背诵它来彰显自己非凡的记忆力。但是因为它太长，而且这种比赛的组织者显然很不通情理，要求背诵的时候不能停顿，所以选手们非得要穿着尿不湿才行。

当然，背诵圆周率和穿不穿尿不湿，对于本书来说不太重要，重要的是这个无穷无尽的圆周率，可以用简单的加减乘除来进行推算。这个推算的过程，和前面那道题一样，也是几个简单步骤的无数次重复，用机器来做很快就能得出结果。要是人来算的话，不知道要算到猴年马月，也许几代人的时间加到一起也算不出来。

再比如说你用计算机听歌，尽管你每次播放的歌曲不同，但是在计算机的内部，播放器总是在按已经设定好的、相同的方式进行播放。在气象部门里，要推算最近几天的天气情况，需要借助于数学工具。也就是说，天气是“算”出来的，气象工作者称之为“数值预报”，即数值天气预报和数理统计预报。它们都是一些复杂的数学方程式，涉及大量的数据和算术运算。这个计算过程每次都一样，唯一不同的是每次参与运算的数据是不同的，这很好理解，因为每天的气压、温度、湿度、风力、风向和云层数据都是不一样的。以前靠手工计算，往往需要很长时间，好不容易算出来了，那天早已过去，天气预报也就成了“天气后报”。现在好

了，借助于计算机，很快就能知道结果。

所以，认为我们现在的计算机都非常智能，而且非常聪明，那只是一种错觉，真实的情况是，所有的步骤都是已经事先安排好的，而要让它干的事情也都事先经过了安排，否则它不会知道如何应付。对此，一个简单的例子就是当你在计算机上写文章的时候，如果输入了一个错字，想删掉它，可以按退格键。如果你按了其他键，则不会达到预期的删除效果，因为当初在编写能够让你写文章的指令时，就是这么安排的，你要么照规矩执行，要么给自己带来麻烦。

看起来我们所要做的是制造一台新的、能够以不变应万变的、计算各种复杂数学题的机器。当然，还得是自动地干这种事情。像这样的机器，因为它能应付各种各样的运算，所以被称为通用计算机。